PRANVEER SINGH INSTITUTE OF TECHNOLOGY

InternsElite

MAJOR PROJECT REPORT

ON

SMART FAKE NEWS DETECTION SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

Branch: Computer Science and Engineering (Artificial Intelligence & Machine Learning)

SUBMITTED BY

Name: NIKHAR
Roll Number: 2401641530123
Section: AIML_III-A

SUBMITTED TO

Department of Computer Science and Engineering (AIML)
Pranveer Singh Institute of Technology
InternsElite

SESSION: 2026–2027 | SEPTEMBER BATCH

CERTIFICATE

This is to certify that the project report entitled **“Smart Fake News Detection System”** is submitted by **NIKHAR** (Roll Number: **2401641530123**), Section **AIML_III-A**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence & Machine Learning) at Pranveer Singh Institute of Technology, under the project programme InternsElite.

Project Guide / Mentor	Head of Department	External Evaluator
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Note: The guide/mentor name and official signatures should be completed by the institution before physical submission.

DECLARATION

I hereby declare that the project work presented in this report entitled **“Smart Fake News Detection System”** is my project work carried out as part of the Bachelor of Technology programme. The explanations, implementation description, results and limitations have been documented to the best of my knowledge. Wherever external tools, libraries, datasets or references have been used, they are acknowledged in this report.

Place: _____
Date: _____

Signature of Student
NIKHAR
Roll Number: 2401641530123

ACKNOWLEDGEMENT

I express my sincere gratitude to Pranveer Singh Institute of Technology and the Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning) for providing the academic environment and opportunity to undertake this major project. I also acknowledge the support and guidance provided through the InternsElite project programme.

I am thankful to the faculty members, mentors and everyone who provided suggestions during the planning, development and testing of the Smart Fake News Detection System. Their guidance helped me understand the practical application of Natural Language Processing, machine learning, web development and database integration.

I also acknowledge the developers and maintainers of Python, Flask, Pandas, scikit-learn, SQLite and other open-source tools used in the project. Finally, I thank my family and friends for their encouragement throughout the work.

NIKHAR
Roll Number: 2401641530123

ABSTRACT

The rapid growth of online news platforms and social media has made information accessible at an unprecedented scale. At the same time, misleading and fabricated news can spread quickly and influence public opinion. Manually checking every article is time-consuming, which creates a need for tools that can assist users in examining news content.

The **Smart Fake News Detection System** is a machine-learning-based web application that analyzes the textual content of a news article and predicts whether it resembles the REAL or FAKE examples learned during training. The system is implemented in Python using Natural Language Processing (NLP), TF-IDF text vectorization and Logistic Regression. A Flask web interface allows a user to submit an article and view the predicted label with a confidence score. SQLite is used to store prediction history so that earlier checks can be reviewed.

The training dataset used in the project contains 44,898 articles, comprising 23,481 FAKE and 21,417 REAL records. The reported random holdout evaluation achieved 98.88% accuracy, with a confusion matrix of [[3394, 67], [19, 4219]] for the class order FAKE, REAL. The application presents three user-facing outcomes: REAL, FAKE and NEEDS VERIFICATION. The third outcome is a cautious interface state for predictions that do not meet the configured decision thresholds; it is not an independent class learned from the dataset.

The project is intended as an educational decision-support prototype, not as a definitive fact-checking authority. Its predictions reflect patterns in the training data and may be affected by topic, source, writing style, time period and dataset coverage. Independent evaluation on diverse, current and verified articles is required before any real-world deployment.

TABLE OF CONTENTS

Chapter	Title
—	Certificate, Declaration, Acknowledgement and Abstract
1	Introduction and Problem Statement
2	Project Objectives and Scope
3	Literature Review and Background
4	System Requirements and Feasibility
5	Dataset Description and Preparation
6	System Analysis and Architecture
7	Methodology and Algorithm
8	Implementation Details
9	Testing, Evaluation and Results
10	Limitations, Security and Ethical Considerations
11	Conclusion and Future Scope
—	References and Appendix

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency–Inverse Document Frequency
LR	Logistic Regression
UI	User Interface
DB	Database
EDA	Exploratory Data Analysis
CSV	Comma-Separated Values

CHAPTER 1: INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

Digital platforms have changed the way people create, publish and consume news. News articles can reach large audiences within minutes through websites, messaging applications and social networks. This speed is useful for communication, but it also makes it easier for inaccurate, misleading or fabricated information to circulate before readers can verify it.

Fake news generally refers to information presented in a news-like format that is false, misleading or fabricated. It may imitate the structure and language of genuine reporting. Distinguishing it from authentic reporting can require checking the publisher, date, evidence, quotations and supporting sources. Text classification can help identify linguistic patterns associated with examples previously labelled as real or fake.

The project applies supervised machine learning to the text of news articles. During training, a model learns statistical relationships between words and the known labels in a prepared dataset. During prediction, the application converts submitted text into numerical features and uses the trained classifier to produce a label and probability estimates.

1.2 Problem Statement

A reader may encounter a news article without knowing whether its content is trustworthy. Manual verification requires time and access to credible independent sources. The project addresses the narrower technical problem of building a web-based text classifier that can rapidly compare an article's language patterns with labelled training examples and present an understandable result.

The system does not independently establish whether an event happened. It does not browse the web, authenticate the publisher, inspect source documents or corroborate claims. Therefore, the problem is framed as **news-text classification**, rather than complete automated fact verification.

1.3 Motivation

- To demonstrate a practical application of NLP and supervised machine learning.
- To provide a simple interface for submitting news text and viewing a prediction.
- To retain previous predictions in a local database for demonstration and review.
- To communicate uncertainty rather than force every input into a confident binary answer.

1.4 Proposed Solution

The proposed solution combines a TF-IDF vectorizer and Logistic Regression classifier inside a scikit-learn pipeline. A Flask application receives text from an HTML form, validates the input, obtains class probabilities and applies configured thresholds. The resulting REAL, FAKE or NEEDS VERIFICATION label and confidence score are displayed on the page. Each prediction is stored in SQLite with the submitted text and timestamp.

1.5 Organization of the Report

The report introduces the problem and objectives, reviews the technical background, describes requirements and data preparation, explains the architecture and methodology, documents implementation and testing, discusses limitations, and concludes with future improvements.

CHAPTER 2: PROJECT OBJECTIVES AND SCOPE

2.1 Objectives

- Build a machine-learning pipeline to classify news text using labelled examples.
- Prepare and clean the news dataset before training.
- Convert text into numerical features using TF-IDF.
- Train a Logistic Regression classifier for FAKE and REAL labels.
- Evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
- Develop a Flask-based interface for submitting news articles.
- Display the predicted class and model confidence information.
- Store prediction history using SQLite.
- Provide a NEEDS VERIFICATION outcome when neither configured class threshold is met.
- Document the limitations of dataset-based text classification.

2.2 Scope of the Project

The current scope includes English-language text input, a locally hosted Flask application, a trained binary classifier, probability-based decision rules and local prediction history. The model accepts article text and does not require the user to provide a URL or publisher metadata.

2.3 Out of Scope

- Automatic web search or live source verification.
- Image, video, audio or social-media account authenticity analysis.
- Multilingual classification beyond the language represented in the training data.
- Publisher reputation scoring or citation verification.
- Production-scale concurrent hosting, user authentication and cloud database deployment.
- Guaranteeing that a REAL prediction means every claim in the article is factually correct.

2.4 Intended Users

The prototype is intended for students, educators and users who want to explore how machine-learning text classification works. It can be used for classroom demonstrations, experimentation and preliminary screening of text. A user should verify important claims using independent, reliable sources.

CHAPTER 3: LITERATURE REVIEW AND TECHNICAL BACKGROUND

3.1 Text Classification

Text classification is the task of assigning one or more predefined categories to a piece of text. Examples include spam detection, sentiment analysis, topic classification and document routing. In supervised classification, a model is trained using examples for which the correct category is already known.

3.2 Natural Language Processing

Natural Language Processing is a field of Artificial Intelligence concerned with computational processing of human language. A typical text-classification workflow may include text cleaning, tokenization, feature extraction, model training and evaluation. The exact preprocessing choices affect which signals are retained by the model.

3.3 TF-IDF Feature Extraction

TF-IDF represents a document using numerical weights for terms. Term frequency reflects how often a term appears in a document, while inverse document frequency reduces the weight of terms that occur in many documents. The resulting sparse vectors allow conventional machine-learning algorithms to process text.

For this project, the vectorizer is configured with English stop-word removal, a maximum vocabulary size of 100,000 features, unigrams and bigrams, a minimum document frequency of 2, a maximum document frequency of 0.95 and sublinear term-frequency scaling.

3.4 Logistic Regression

Logistic Regression is a supervised classification algorithm. Although its name contains the word regression, it is commonly used for classification. It computes a weighted combination of input features and maps the result to class probabilities. In this project, it distinguishes two training labels: FAKE (0) and REAL (1).

3.5 Related Approaches

Common approaches to text classification include Naive Bayes, Support Vector Machines, Logistic Regression, decision trees and neural networks. TF-IDF combined with linear classifiers is a practical baseline because it is computationally efficient, interpretable at the feature-weight level and suitable for sparse text representations. More advanced contextual language models may capture semantic relationships that simple bag-of-words representations miss.

3.6 Difference Between Classification and Fact-Checking

A text classifier learns correlations between the words and labels in its training corpus. Fact-checking requires examining claims against evidence, checking dates and context, and consulting independent sources. A classifier can therefore label a true article as FAKE or a fabricated article as REAL, especially when the input differs from its training distribution. The distinction is central to interpreting the results of this project.

CHAPTER 4: SYSTEM REQUIREMENTS AND FEASIBILITY

4.1 Hardware Requirements

Component	Minimum / Practical Requirement
Processor	Modern dual-core processor or better
RAM	8 GB recommended for comfortable local development
Storage	At least 1 GB free space for code, dataset, model and outputs
Display	Standard desktop/laptop display
Network	Internet connection for obtaining dependencies and accessing the repository

4.2 Software Requirements

Software / Library	Purpose
Python 3	Programming language and runtime
Flask	Web application framework
scikit-learn	TF-IDF vectorizer, Logistic Regression and evaluation
Pandas	Dataset loading and data preparation
Joblib	Saving and loading the trained model
SQLite	Local prediction-history database
HTML / CSS	Web interface
VS Code	Development environment
Git and GitHub	Version control and source-code hosting

4.3 Functional Requirements

- The system shall display a home page containing a news-text input form.
- The system shall reject text shorter than the minimum configured word count.
- The system shall obtain FAKE and REAL probability estimates from the trained model.
- The system shall apply configured thresholds and return one of three display labels.
- The system shall display the result and confidence percentage.
- The system shall save each valid prediction in the local database.
- The system shall provide a history page listing stored predictions.

4.4 Non-Functional Requirements

- Usability: the interface should be straightforward and understandable.
- Performance: a prediction should be returned quickly for ordinary article-sized input on a local machine.
- Maintainability: training, database and web application code are separated into modules.

- Reliability: invalid short input should produce a helpful validation message.
- Transparency: the system should communicate that its output is a model prediction, not proof of truth.

4.5 Feasibility Study

Technical feasibility: The project uses mature Python libraries and can run on a standard development computer. **Economic feasibility:** The core software stack is open source, so no paid software licence is required for the prototype. **Operational feasibility:** A simple web form and history page make the system usable for demonstration. **Schedule feasibility:** The modular workflow allows dataset preparation, model training, web integration and testing to be completed as separate tasks.

CHAPTER 5: DATASET DESCRIPTION AND PREPARATION

5.1 Dataset Overview

The project uses the Fake and Real News Dataset in CSV form. The working dataset contains 44,898 news records. The reported class counts are 23,481 FAKE articles and 21,417 REAL articles. The project directory includes the original Fake.csv and True.csv files and prepared combined/cleaned CSV files.

Dataset measure	Value
Total articles	44,898
FAKE articles	23,481
REAL articles	21,417
FAKE share	Approximately 52.30%
REAL share	Approximately 47.70%

5.2 Data Fields

The source files contain news article information such as title, text, subject and date. The model training workflow uses the article text and its binary label. The application accepts article text, so a text-only training pipeline is used to keep training input consistent with inference input.

5.3 Data Cleaning

- Load the CSV data using Pandas.
- Remove records with missing text or labels.
- Convert article text to string form and trim leading/trailing whitespace.
- Remove records with very short text according to the training script's minimum-length rule.
- Remove duplicate text records.
- Normalize labels so FAKE maps to 0 and REAL maps to 1.
- Separate the text column as X and the label column as y.

5.4 Train-Test Split

The dataset is divided into training and testing subsets using an 80:20 split. Stratification is used to preserve the class distribution in both subsets, and random_state=42 makes the split reproducible. The test subset is held out from model fitting and used for the reported evaluation.

5.5 Dataset Limitations

The dataset reflects a particular collection period, set of publishers and writing styles. It may contain repeated topics, source-specific language patterns or historical reporting conventions. Random train-test evaluation does not establish performance on current news, unseen publishers or entirely different topics. Dataset coverage must be considered when interpreting a prediction about a subject that is not represented in the training examples.

CHAPTER 6: SYSTEM ANALYSIS AND ARCHITECTURE

6.1 Overall Architecture

The application follows a simple layered workflow. A user submits text through the browser. Flask receives and validates the request. The saved scikit-learn pipeline transforms the text with TF-IDF and computes class probabilities through Logistic Regression. The decision layer applies the thresholds and sends the result to the HTML template. The prediction is also stored in SQLite.



High-level workflow of the proposed system

6.2 Main Modules

Module	Responsibility
src/prepare_data.py	Prepare and clean source CSV files.
src/train_model.py	Train, evaluate and save the classification pipeline.
outputs/fake_news_model_improved.pkl	Serialized TF-IDF and Logistic Regression pipeline.
src/database.py	Create SQLite table, save predictions and retrieve history.
app.py	Flask routes, input validation, inference and result rendering.
templates/index.html	Home page and prediction form/result.
templates/history.html	Prediction history display.
static/	Stylesheets and static assets, where present.
screenshots/	Project demonstration screenshots.

6.3 Data Flow

- Input: user enters an English news article in the browser.
- Validation: the Flask route checks that the submitted text contains at least 10 words.
- Vectorization: the saved TF-IDF transformer converts text into a sparse feature vector.
- Classification: Logistic Regression returns probability estimates for FAKE and REAL.
- Decision: threshold rules map the probabilities to REAL, FAKE or NEEDS VERIFICATION.
- Persistence: the text, label, confidence and timestamp are saved in SQLite.
- Output: the result is displayed on the home page and can be viewed in history.

6.4 Database Design

The SQLite database is stored at outputs/news_history.db. The predictions table contains an automatically incrementing id, news_text, prediction, confidence and created_at. This structure is sufficient for a local prototype and can support chronological retrieval of prior predictions.

Field	Type / Constraint	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique prediction identifier
news_text	TEXT NOT NULL	Submitted article text
prediction	TEXT NOT NULL	REAL, FAKE or NEEDS VERIFICATION
confidence	REAL NOT NULL	Maximum of the two class probabilities, expressed as a percentage
created_at	TEXT NOT NULL	Local timestamp when the prediction is stored

CHAPTER 7: METHODOLOGY AND ALGORITHM

7.1 Proposed Workflow

The workflow begins with dataset preparation and label normalization. The text-only feature column and binary target are split into training and testing data. A scikit-learn Pipeline ensures that the same TF-IDF transformation is applied during training and prediction. Logistic Regression is fitted on the training set and evaluated on the held-out test set. The trained pipeline is serialized and loaded by the Flask application.

7.2 TF-IDF Configuration

Parameter	Configured value
stop_words	english
max_features	100000
ngram_range	(1, 2)
min_df	2
max_df	0.95
sublinear_tf	True

7.3 Logistic Regression Configuration

Parameter	Configured value
max_iter	2000
C	1.0
class_weight	balanced
random_state	42

7.4 Decision Rule in the Web Application

The classifier has two learned classes: 0 for FAKE and 1 for REAL. The web application converts the corresponding probabilities to percentages and applies separate thresholds. The configured REAL threshold is 0.60 and the FAKE threshold is 0.80. If the REAL probability reaches its threshold, the displayed label is REAL. Otherwise, if the FAKE probability reaches its threshold, the displayed label is FAKE. If neither condition is met, the displayed label is NEEDS VERIFICATION.

Condition	Displayed outcome
REAL probability ≥ 0.60	REAL
Otherwise, FAKE probability ≥ 0.80	FAKE
Neither threshold is met	NEEDS VERIFICATION

The ordering of the conditions is part of the implementation: the REAL condition is checked first. The displayed confidence is the maximum of the two class probabilities. These values are model estimates and should not be interpreted as calibrated probabilities unless calibration has been separately tested.

7.5 Pseudocode

```
LOAD saved model pipeline
READ news text from form

IF word_count(news text) < 10:
    SHOW validation error
ELSE:
    probabilities = model.predict_proba(news text)
    fake_probability = probability for class 0
    real_probability = probability for class 1

    IF real_probability >= 0.60:
        label = REAL
    ELSE IF fake_probability >= 0.80:
        label = FAKE
    ELSE:
        label = NEEDS VERIFICATION

    confidence = max(fake_probability, real_probability) * 100
    SAVE text, label, confidence and timestamp
    DISPLAY label and confidence
```

CHAPTER 8: IMPLEMENTATION DETAILS

8.1 Development Environment

The project is developed in Python using Visual Studio Code on a Windows environment. PowerShell is used to run scripts and launch the Flask server. Git is used for version control, and the source code is hosted in a GitHub repository.

8.2 Dataset Preparation Module

The preparation module reads the source CSV files, cleans text and labels, and creates a consistent dataset for training. Keeping data preparation separate from model training makes the workflow easier to reproduce and maintain.

8.3 Model Training Module

The training module loads the cleaned dataset, removes invalid or duplicate examples, normalizes the binary labels and creates a stratified 80:20 split. It then fits a pipeline containing TfidfVectorizer and LogisticRegression. The script prints evaluation metrics and saves the complete pipeline to outputs/fake_news_model_improved.pkl using Joblib.

8.4 Flask Application

The Flask application loads the serialized pipeline when the application starts and initializes the prediction-history table. The home route handles GET and POST requests. On POST, the route reads the news field, validates its length, calls predict_proba and applies the decision thresholds. The prediction result, confidence and submitted text are passed to the home template.

8.5 Database Module

The database module uses Python's sqlite3 library. The create_table function creates the predictions table if it does not already exist. The save_prediction function inserts the submitted text, result, confidence and timestamp using parameterized SQL. The get_history function retrieves records in descending id order.

8.6 User Interface

The user interface provides a text area for news input and a submit action. After submission, it displays the result label and confidence. A separate history page displays previously stored predictions. The project screenshots should be included in the final repository to demonstrate the actual running interface.

8.7 Repository

The source repository is hosted at: https://github.com/nikhar-10/Smart_Fake_News_Detection. The repository is intended to contain the application source, model-related files, dataset/preparation assets, templates, screenshots, requirements and project documentation. The final repository should be checked to ensure that all referenced files are present and that large data files are handled appropriately.

CHAPTER 9: TESTING, EVALUATION AND RESULTS

9.1 Testing Strategy

Testing was carried out at the model and application levels. Model evaluation used a held-out test subset. Application testing used manually entered news-like text to check form submission, result display, confidence display and history storage. Manual examples are demonstration tests and do not replace an independently labelled benchmark.

9.2 Model Evaluation

The reported training run achieved an accuracy of 98.88%. The confusion matrix was reported in the class order FAKE, REAL as follows:

Actual \ Predicted	Predicted FAKE	Predicted REAL
Actual FAKE	3,394	67
Actual REAL	19	4,219

From the matrix, 3,394 FAKE examples and 4,219 REAL examples were correctly classified. There were 67 FAKE examples predicted as REAL and 19 REAL examples predicted as FAKE. The high random-split score should be interpreted cautiously because dataset-specific patterns, duplicated or near-duplicated reporting, source style and time-period effects can make a random split easier than a true out-of-distribution test.

9.3 Reported Classification Metrics

Class	Precision	Recall	F1-score
FAKE	0.99	0.98	0.99
REAL	0.98	1.00	0.99

9.4 Application Test Cases

Test	Input / Action	Expected application behavior
TC-01	Submit fewer than 10 words	Display a validation error; do not run prediction.
TC-02	Submit a longer news article	Run model inference and show a result.
TC-03	Submit text with REAL probability ≥ 60%	Display REAL.
TC-04	Submit text with FAKE probability ≥ 80%, when REAL threshold is not met	Display FAKE.
TC-05	Submit text meeting neither threshold	Display NEEDS VERIFICATION.
TC-06	Open history page after predictions	Display saved prediction records.
TC-07	Restart the Flask application	Load the saved model and retain database history.

9.5 Demonstration Observations

During manual demonstration, the user reported that the NASA/Hubble-related test text produced a REAL result, and separate fabricated or ambiguous examples produced FAKE or NEEDS VERIFICATION outcomes. These are observations from the project demonstration, not a controlled independent test set. The exact probability values and article provenance should be preserved with the screenshots if they are used as evidence.

9.6 Interpretation

The model evaluation indicates that the classifier learned patterns that separate the two classes in the selected dataset split. However, a high test accuracy does not guarantee correct classification of new news articles. A model may learn publisher-specific vocabulary, topic frequency or historical writing conventions rather than factual truth. Therefore, the result is a useful machine-learning demonstration but should not be presented as a verified truth score.

CHAPTER 10: LIMITATIONS, SECURITY AND ETHICAL CONSIDERATIONS

10.1 Technical Limitations

- The model is trained on a fixed dataset and may not represent current events.
- The model uses TF-IDF, which emphasizes word and phrase frequency and does not deeply understand context.
- The model does not verify claims against authoritative sources.
- Short, paraphrased, multilingual or highly technical text may produce unreliable results.
- Dataset imbalance, duplicates, publisher patterns and topic leakage can affect evaluation.
- Probability outputs may be overconfident and have not been established as calibrated confidence values.
- The NEEDS VERIFICATION outcome is created by threshold rules; it is not a third class learned during training.

10.2 Ethical Use

The application should not be used to accuse a person, publisher or organization of spreading misinformation solely on the basis of its output. A false FAKE label can harm trust and reputation, while a false REAL label can encourage the spread of inaccurate information. Users should seek corroboration from multiple reliable sources and inspect original evidence before sharing high-impact claims.

10.3 Privacy and Data Handling

The prototype stores submitted news text and prediction metadata in a local SQLite database. If users submit private or sensitive text, that content may remain in the local history database. A production version should provide a clear retention policy, deletion controls, access restrictions and secure handling of stored content.

10.4 Security Considerations

- Keep Flask debug mode disabled in production.
- Validate and limit submitted input size to reduce resource abuse.
- Use parameterized SQL statements for database inserts.
- Do not commit secrets, credentials or private data to the public repository.
- Review dataset and model licensing before redistribution.
- Use dependency versions and security updates appropriate to the deployment environment.

CHAPTER 11: CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

The Smart Fake News Detection System demonstrates an end-to-end machine-learning workflow for classifying news text. It combines dataset preparation, TF-IDF feature extraction, Logistic Regression, model evaluation, a Flask web interface and SQLite-based prediction history. The application provides a simple way to submit text and inspect the classifier's output.

The project achieved a reported 98.88% accuracy on a random held-out split of the prepared dataset. The web application exposes three user-facing outcomes: REAL, FAKE and NEEDS VERIFICATION. The third outcome makes the interface more cautious when neither configured threshold is met. Nevertheless, the model remains a text-pattern classifier and cannot independently confirm the truth of a news claim.

11.2 Future Enhancements

- Evaluate on a time-based split and a separate dataset from unseen publishers.
- Build a manually verified benchmark containing recent science, politics, health, business and technology articles.
- Add source URL and publication date fields and provide source-level context.
- Integrate retrieval from trusted sources for claim corroboration, with citations and transparent evidence.
- Explore contextual transformer models and compare them with the TF-IDF baseline.
- Calibrate probability estimates and select thresholds using validation data and error-cost analysis.
- Add multilingual support with language detection and suitable multilingual models.
- Improve the dashboard with confusion matrix, model metrics and prediction analytics.
- Add user authentication, deployment configuration, logging and database retention controls.

11.3 Final Note

The system should be understood as an educational screening aid. Its output is a prompt for further verification, not a final judgement about the factual accuracy of an article.

REFERENCES

- Fake and Real News Dataset, Kaggle: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset
- scikit-learn documentation, TfidfVectorizer: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- scikit-learn documentation, LogisticRegression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- scikit-learn documentation, model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html
- Flask documentation: https://flask.palletsprojects.com/
- Pandas documentation: https://pandas.pydata.org/docs/
- Python sqlite3 documentation: https://docs.python.org/3/library/sqlite3.html
- Joblib documentation: https://joblib.readthedocs.io/

APPENDIX A: PROJECT DIRECTORY STRUCTURE

```
Smart_Fake_News_Detection/
|
|-- data/
|   |-- Fake.csv
|   |-- True.csv
|   |-- news.csv
|   |-- news_cleaned.csv
|
|-- notebooks/
|   |-- Fake_News_Analysis.ipynb
|
|-- src/
|   |-- prepare_data.py
|   |-- train_model.py
|   |-- database.py
|
|-- outputs/
|   |-- fake_news_model_improved.pkl
|   |-- news_history.db
|
|-- templates/
|   |-- index.html
|   |-- history.html
|
|-- static/
|-- screenshots/
|-- app.py
|-- requirements.txt
|-- README.md
```

Note: This is the project structure described during development. Confirm the actual repository contents before final submission; generated model/database files may or may not be committed depending on repository size and policy.

APPENDIX B: SAMPLE USER WORKFLOW

- Start the application from the project root using: python app.py
- Open the local Flask address shown in the terminal.
- Enter an English news article containing at least 10 words.
- Submit the form and review the displayed label and confidence percentage.

• Open the History page to review saved predictions.

APPENDIX C: GITHUB REPOSITORY

Project repository: https://github.com/nikhar-10/Smart_Fake_News_Detection

END OF REPORT